OrderFit System Design

Oct 14, 2026 · @Sazid

Summary

OrderFit is a four-layer system:

- **Next.js** for the screens.
- **FastAPI** for the API, with Pydantic checking every request and response.
- **LangGraph** for the agent workflow: extract, confirm, decide, draft, approve, negotiate.
- **A pure-Python decision engine** that does all the money arithmetic and contains no AI.

The AI model reads and writes words; the engine decides. The owner approves every outgoing action.

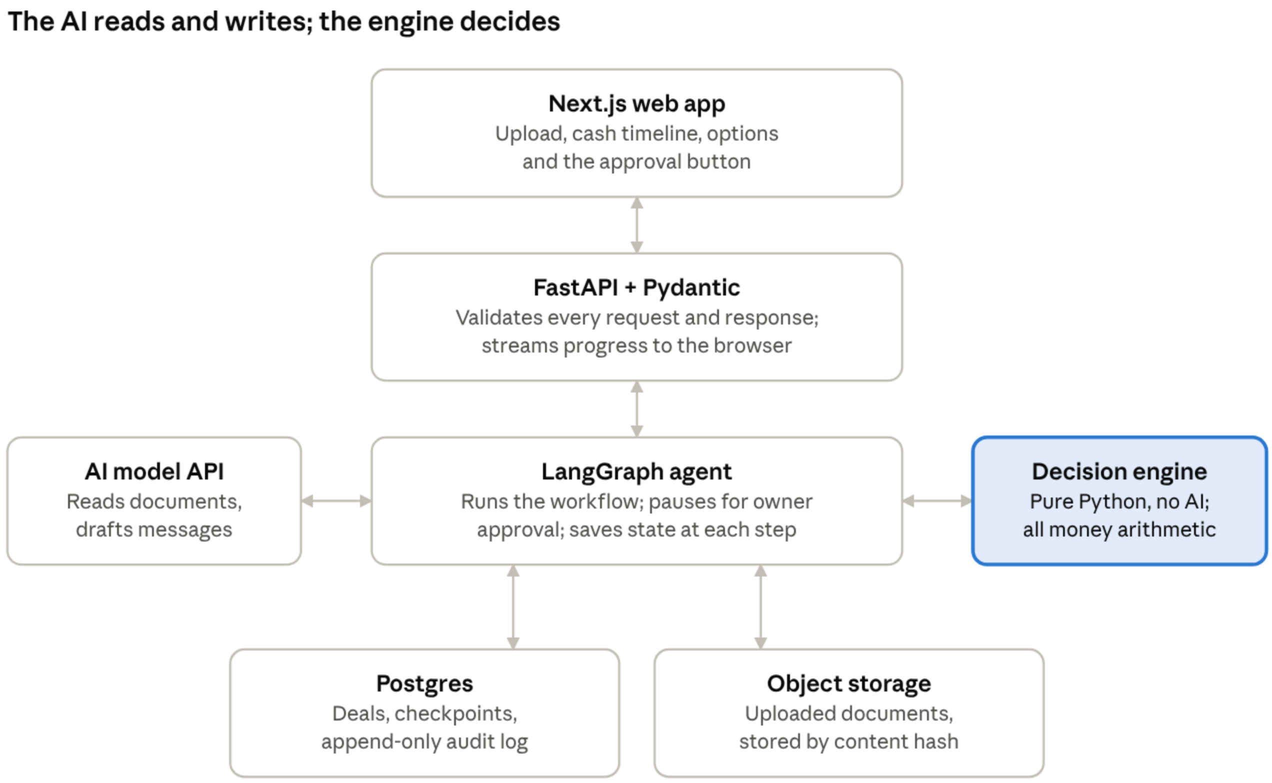
Headline numbers. The pilot figures assume 10,000 businesses running 20 analyses a month each; the derivations are in the capacity section.

Measure	Competition MVP	Pilot (10,000 businesses)
Analyses	A few dozen demo and test runs	200,000 a month; peak about 0.84 a second
Time to recommendation	About 12–17 seconds	Same; dominated by AI calls
Engine compute per analysis	Under 1 CPU-second, worst case	Under one CPU core busy at peak
AI cost per analysis	About $0.09 (assumed prices)	About $17,100 a month, or $1.71 per business
Concurrent analyses at peak	1	About 14
Database growth	Negligible (SQLite)	About 20 GB a month after pruning, plus about 300 GB of documents

Measure	Competition MVP	Pilot (10,000 businesses)
Infrastructure	One app server plus a hosted frontend	Two API instances, two workers, Postgres, object storage, a job queue

The main conclusion: **AI calls set both the speed and the cost, while the engine is cheap.** That justifies running a thorough brute-force stress test on every analysis instead of shortcuts.

Architecture



The AI reads and writes; the engine decides

Next.js web app
Upload, cash timeline, options and the approval button
FastAPI + Pydantic
Validates every request and response; streams progress to the browser
AI model API
Reads documents, drafts messages
LangGraph agent
Runs the workflow; pauses for owner approval; saves state at each step
Decision engine
Pure Python, no AI; all money arithmetic
Postgres
Deals, checkpoints, append-only audit log
Object storage
Uploaded documents, stored by content hash

OrderFit architecture · 4 layers, 2 stores

Requests flow down from the browser to the agent. The agent calls the AI model only for text work (reading documents, explaining, drafting) and calls the engine for every number. The engine never calls the AI.

Component	Responsibility	Key design choices
Next.js web app	Upload documents, show the cash timeline and options, collect approvals	No money logic in the browser; progress streamed from the API
FastAPI + Pydantic	API endpoints: create deal, upload document, confirm figures, approve, submit reply	Pydantic models are the contract between every layer; money typed as `Decimal`; server-sent events for progress
LangGraph agent	Runs the workflow as a state machine; pauses for the owner; resumes after a restart	Checkpoints in SQLite for the MVP, Postgres for the pilot; only two pause points
AI model API	Extraction into fixed schemas, explanations, email drafts, reading negotiation replies	Structured output parsed into Pydantic; low temperature; fixed model version; every call logged
Decision engine	Cash timeline, stress tests, option generation, delay tolerance, loan sizing, ranking	Pure functions, deterministic, versioned; no network calls; unit-tested against the catalogue
Postgres	Deals, cash events, results, checkpoints, approvals	An append-only audit table that can't be edited after writing
Object storage	Original uploaded documents	Stored by content hash, which also detects duplicate uploads (edge case D-09)

Agent workflow

The LangGraph graph has ten nodes and two pause points. It loops back to the engine every time a negotiation reply arrives, capped at three rounds.

1. **Ingest:** store each document, compute its content hash, and reject duplicates.
2. **Extract** (AI): read the order and supplier quote into a fixed `OrderTerms` schema. Each field carries its source text and a confidence level.
3. **Validate** (rules, no AI). Checks that:
    - line items add up to the total;
    - dates parse unambiguously;

   - payment terms have a start date.

   It also scans for injected instructions and for fraud signals such as changed bank details or mismatched domains.

4. **Confirm** (**pause 1**): the owner checks the extracted figures and answers any ambiguity, such as which date "Net 30" counts from.
5. **Compute** (engine): timeline, options, stress tests, delay tolerance, loan sizing and ranking.
6. **Explain** (AI): write the recommendation in plain language.
7. **Draft** (AI): write the message for the chosen option.
8. **Approve** (**pause 2**): the owner approves, edits or rejects. The decision is logged.
9. **Send:** in the MVP the send is simulated. Each send carries a unique key, so a retry never sends twice.
10. **Read reply** (AI): extract only the proposed terms from the counterparty's reply, then return to step 5.

Two guards that make the AI safe to use

- **Number guard.** The AI never types a figure. Explanations and drafts use placeholders such as `{lowest_balance}`, which are filled from the engine's output. A final check confirms that every number in the text matches an engine value, and blocks the message if any number doesn't.
- **Terms guard.** Any loan rate, fee or date in a draft must match the financing catalogue or the confirmed deal terms. Anything else is blocked and logged; this covers edge case X-06.

State saved at every step

Field	Contents
`deal`	Business profile, customer record, confirmed order terms
`events`	Every dated cash movement used by the engine
`result`	Engine output: timeline, options, stress results, recommendation
`round`	Negotiation round number (stops at 3)

Field	Contents
`pending_action`	The draft awaiting approval, if any
`flags`	Ambiguities, fraud and injection warnings, owner overrides

The checkpoint saves this state after every step. A server restart during the demo resumes from the last step instead of starting over.

Decision engine

The engine is small, exact and exhaustive. It tries every option against every delay instead of using clever shortcuts, because the shortcuts give wrong answers on exactly the cases that matter.

Money representation

Store every amount as an **integer number of the smallest unit** (cents or paisa). $9,370.00 is stored as `937000`. Integers are exact and fast, and they work with numpy arrays. `Decimal` is used only at the API boundary for display and input. This avoids the rounding errors in edge case X-04 without slowing the engine.

Algorithms

Step	Method	Notes
Daily balance	Sort events by day, outflows before inflows; take a running sum over the horizon	Records lowest balance, first breach day and every breach interval
Option effect	Each option is a change applied to the event list (move a payment, add a loan draw and repayment, scale a receipt)	Options combine; up to 5 levers gives at most 31 combinations
Stress test	Shift the order's receipt by *d* days; optional partial payment, cancellation or discount	One run per option per delay

Step	Method	Notes
Delay tolerance	Scan *d* = 0, 1, 2 … 180 and **stop at the first failure**	See below
Loan sizing	Required loan = reserve minus the lowest balance while the loan is outstanding, then check that repayment itself is survivable	Rina: $5,000 − (−$4,000) = **$9,000**, matching the hand calculation
Ranking	Options that meet the target tolerance are ranked by cost, then by amount borrowed, then by a fixed option order. Options that don't are ranked below them, by tolerance.	"Safe enough, then cheapest"

Why not binary search on delay

Binary search assumes that once a plan fails at some delay, it fails at every longer delay. Cash timelines break that assumption. A receipt that lands just after a loan repayment or a payroll date can fail at 21 days late and pass again at 26, because a different inflow arrives in between.

A linear scan that stops at the first failure is always correct, and at 181 steps it is cheap. Define delay tolerance as **"the last delay before the first failure"**, never "the largest delay that passes".

Ranking: tolerance first, but capped

Ranking purely by tolerance would favour an expensive plan that survives 90 days over a cheap one that survives 30 days, even when 30 days is all the owner needs. So set a **target tolerance**: 15 days by default, 30 for weakening customers, 60–90 in downturn mode, following the trust ladder. Every option that meets the target then competes on cost.

Compute cost

Worst case per analysis: 31 option combinations × 181 delays = **5,611 timeline runs**.

Implementation	Work per run	Estimated total
Pure Python loop over ~200 events	~200 operations	About 0.2–0.5 seconds
numpy: baseline daily array plus a change array per option, then a minimum	One pass over 181 days	About 5–20 milliseconds

These are estimates; measure on your own machine. Either way, the engine is under 1 CPU-second, against 12–15 seconds of AI calls. **Start with the pure-Python version** because it is easier to read and to test, and move to numpy only if profiling shows a need.

Data model

Eight entities. The Pydantic models below are the single contract shared by the API, the agent and the engine.

Entity	Key fields	Notes
Business	cash today, reserve, currency, payroll schedule	One per account
Customer	name, payment history, trust level, red flags	The trust ladder reads from here
Deal	business, customer, order terms, status, negotiation round	One per order decision
CashEvent	day, amount (minor units), direction, kind, source document and text	Every number traceable to its source
FinancingOffer	rate per month, fee, minimum, maximum, term	Fixed catalogue; the AI can't add rows
Result	timeline, options, stress table, delay tolerance, recommendation, engine version	Stored so any result can be reproduced
Approval	deal, action, approver, decision, time	Append-only
AuditEntry	actor (owner, agent, engine), event, details, time	Append-only; feeds the security report

Core schemas

```
from datetime import date
from enum import Enum
from pydantic import BaseModel, Field

class Direction(str, Enum):
    IN = "in"
    OUT = "out"

class SourceRef(BaseModel):
    document_hash: str
    quote: str                      # the exact text the figure came from
    confidence: float = Field(ge=0, le=1)

class CashEvent(BaseModel):
    day: int = Field(ge=0, le=365)
    amount_minor: int = Field(gt=0)  # cents or paisa, always positive
    direction: Direction
    kind: str                        # payroll, deposit, receipt, loan_draw
...
    source: SourceRef | None = None

class OrderTerms(BaseModel):
    order_value_minor: int = Field(gt=0)
    currency: str = Field(min_length=3, max_length=3)
    delivery_date: date
    payment_days: int = Field(ge=0, le=365)
    payment_basis: str | None        # "invoice" or "delivery"; None means
ambiguous: ask
    cancellation_clause: str | None
    late_penalty_pct_per_day: float | None
    sources: dict[str, SourceRef]    # one per extracted field

class OptionResult(BaseModel):
    option_id: str
    lowest_balance_minor: int
    first_breach_day: int | None
    delay_tolerance_days: int | None  # None means safe even with no payment
    cost_minor: int
    borrowed_minor: int
    meets_target: bool
```

Three details matter:

- `payment_basis: None` forces the confirm step to ask about ambiguous terms (D-01) instead of guessing.
- `delay_tolerance_days: None` represents "safe even with no payment" (X-01). The display layer shows "fails even on time" when the value is negative (X-02).
- `amount_minor` is always positive, with `direction` carrying the sign. This prevents a whole class of sign errors.

Capacity calculations

Every figure below follows from the stated assumptions. Change an assumption and the arithmetic shows what moves. The AI prices are **assumptions** ($3 per million input tokens, $15 per million output tokens), not quotes; check your provider's current price page before using these numbers in the pitch.

Load

Quantity	Calculation	Result
Analyses per month	10,000 businesses × 20	**200,000**
Per working day	200,000 ÷ 22	9,091
Average per second, business hours	9,091 ÷ (9 h × 3,600 s)	0.28
Peak per second	0.28 × 3 (peak-hour factor)	**0.84**

AI usage and cost per analysis

Call	Input tokens	Output tokens
Extract the order	4,000	800
Extract the supplier quote	4,000	800
Explain the recommendation	3,000	500
Draft the message	2,000	400
Read one negotiation reply (average one round)	1,500	300

Call	Input tokens	Output tokens
Total	**14,500**	**2,800**

- Input cost: 14,500 × $3 ÷ 1,000,000 = **$0.0435**
- Output cost: 2,800 × $15 ÷ 1,000,000 = **$0.0420**
- **Per analysis: $0.0855, about 9 cents**
- Per month at pilot scale: 200,000 × $0.0855 = **$17,100**
- Per business per month: 20 × $0.0855 = **$1.71**

For perspective, in the Rina case one analysis protects $12,630 of profit and avoids a missed payroll. Two levers cut the cost further: a smaller model for extraction, and caching the fixed instruction text that repeats on every call.

Throughput against provider limits

- Tokens at peak: 0.84 analyses/s × 17,300 tokens = 14,532 tokens/s ≈ **872,000 tokens a minute**.
- Requests at peak: 0.84 × 5 calls = 4.2 a second ≈ **252 requests a minute**.

Confirm that your provider account tier allows this before a pilot. The MVP is far below any limit.

Latency to recommendation

This assumes output at about 100 tokens a second plus about 1 second to the first token. The owner's confirmation time is not included.

Step	Estimate
Upload and hashing	0.5 s
Two extractions in parallel (800 tokens each)	~9 s
Validation	< 0.1 s
Engine (worst case)	< 1 s
Explanation (500 tokens)	~6 s
Total	**about 12–17 s**

The draft message adds about 5 seconds after the recommendation appears. Stream progress to the screen so the wait reads as work, not a freeze.

Concurrency

Little's law: concurrent analyses = arrival rate × time in system = 0.84 × 17 s ≈ **14 at peak**. Each mostly waits on the AI, so an async FastAPI service handles this easily. Two API instances and two workers give redundancy, not capacity.

Storage per deal

Item	Size	Notes
Documents (3 × ~500 KB)	1.5 MB	Object storage
Deal, events, result	~50 KB	Postgres
Workflow checkpoints (~25 steps × 20 KB)	~500 KB	Prune to the final state when a deal closes: ~20 KB
Approvals and audit log	~30 KB	Never pruned

- Database per deal: ~580 KB before pruning, **~100 KB after**.
- Database per month: 200,000 × 100 KB = **20 GB**. Without pruning it would be 116 GB, about 6× more, so pruning matters.
- Documents per month: 200,000 × 1.5 MB = **300 GB**, or 3.6 TB for a year's retention.

Pilot infrastructure

Part	Size
API instances	2 × (2 vCPU, 4 GB)
Workflow workers	2 × (2 vCPU, 4 GB)
Postgres (managed)	2 vCPU, 8 GB, 500 GB disk
Queue (Redis)	Smallest managed tier
Object storage	Pay per GB, with a retention policy

Competition budget

Engine tests make no AI calls and cost nothing. An end-to-end evaluation of 30 scenarios × 3 runs each = 90 runs × $0.0855 ≈ **$7.70**. Allow $30–50 of AI spend for the whole week, including development.

Security, reliability and observability

Security: four layers

Layer	Control	Edge cases covered
Permissions	Three tiers. *Read*: extract and analyse. *Suggest*: draft messages. *Act*: send, only after approval. The agent never holds money-moving permission.	O-06
Input	Document text is wrapped and labelled as data. An injection scanner flags imperative text aimed at the system. Bank-detail changes and domain mismatches raise fraud flags.	D-05, D-12, F-01, F-02
Output	The number guard and the terms guard (see Agent workflow) block any figure or loan term the engine or catalogue didn't produce.	X-06
Record	An append-only audit log of every AI call, engine run, override and approval. Documents are stored by hash, and the engine version is saved with each result.	O-01, O-03

Data protection: TLS in transit; encryption at rest for the database and documents; secrets in environment variables, never in code; personal names and bank numbers masked in logs.

Reliability

Failure	Response
AI call times out	Retry twice with growing waits (1 s, then 4 s). Then fall back to manual entry; the engine still runs (X-05).
AI returns output that fails the schema	Re-ask once with the validation error. Then mark the field as needing owner input.
Server restarts mid-deal	LangGraph resumes from the last checkpoint.
Duplicate request (double click, retry)	An idempotency key on every write and send.
Engine bug	The engine version is stored with each result, so any result can be re-run and compared after a fix.

Observability

- **Traces:** one trace per deal, covering every node, AI call, duration and token count. LangSmith or OpenTelemetry both work.
- **Metrics:**
    - time to recommendation (median and slowest 5%);
    - AI cost per analysis;
    - schema-failure rate;
    - guard blocks;
    - owner override rate.
- **Evaluation run:** the catalogue's cases run nightly. Engine cases must match exactly. End-to-end cases check that the recommendation and the flags match the expected behaviour.

The owner override rate is the most telling product metric. If owners often reject the recommendation, the trust levels or targets need tuning.

MVP versus production

With six days left, build the parts judges score and fake nothing. Everything deferred below is a named next step in your technical document, not a hidden gap.

Area	Build this week (MVP)	Defer to the pilot
Database	SQLite for deals and checkpoints	Managed Postgres
Documents	Local folder, stored by hash	Object storage with retention
Jobs	FastAPI background task	Redis queue and separate workers
Sending	Simulated send, shown in the audit log	Real email integration
Customer history	Seeded in the demo data	Imported from accounting software
Engine	Pure Python, all 6 algorithms, Rina cases exact	numpy speed-up if profiling demands it
Guards	Number guard, terms guard, injection flag, approval pauses	Full fraud model
Observability	Structured logs plus a JSON audit export	Tracing, dashboards, nightly evaluation
Deployment	One API host plus Vercel	Two of each, with health checks

Repository layout

```
orderfit/
  engine/        timeline.py  options.py  stress.py  rank.py   (no imports from agent/ or api/)
  agent/         graph.py  nodes/  guards.py  prompts/
  api/           main.py  routes/  schemas.py
  web/           Next.js app
  tests/         engine/ (catalogue cases, exact)  e2e/ (with AI)
  data/          rina_demo.json  financing_catalogue.json
```

Build order

- ☐ **14 Oct:** deploy "hello world" for the API and the frontend; build the engine timeline and the Rina cases R-01 to R-06 passing.
- ☐ **15 Oct:** stress test, delay tolerance, loan sizing and ranking (R-07 to R-09 passing). Decide Next.js or Streamlit by evening.

- ☐ **16 Oct:** LangGraph flow with extraction, the confirm pause, the engine call and the explanation; number guard.
- ☐ **17 Oct:** draft, the approve pause, simulated send, reply reading and the negotiation loop (R-10); terms guard and injection flag.
- ☐ **18 Oct:** screens finished; catalogue core cases run end to end; plain-model baseline comparison recorded.
- ☐ **19 Oct:** technical document, security self-assessment, deck, demo video; submit.
- ☐ **20 Oct:** buffer for fixes only. The deadline is 23:59 Beijing time, which is 21:59 in Dhaka.